A Strategic & Technical Reference for Enterprise AI Agents, Fixed-Seat Subscriptions, and Telemetry Integration

Audience: Executive & Technical Stakeholders | Scope: M365, Copilot Studio, Azure AI Foundry, Fabric | Status: Verified Technical Specification

Executive Overview

This document provides a comprehensive, research-backed reference on how AI agents and seat licenses operate across the Microsoft enterprise ecosystem.

It covers:
1. **The 4 Primary Agent Pathways in Microsoft**: Capabilities, output metrics, admin access, and exposed APIs.
2. **Enterprise AI Subscriptions (Fixed Seats)**: GitHub Copilot and Microsoft 365 Copilot tiers, pricing, and extractable telemetry fields.
3. **Practical Implementation Strategy With Current Resources**:
* **Fixed Seats Telemetry**: Obtaining real employee adoption and license efficiency data via Company Admin (Balaji).
* **Custom Autonomous Agents**: Code-first telemetry observability using Azure AI Foundry / Azure OpenAI.
4. **How to Identify Stakeholder GitHub Repositories**: Inspecting folder structures to distinguish declarative, low-code, and code-first agents.

1. The 4 Microsoft Agent Pathways (Telemetry, APIs & Setup)

Pathway 1: Declarative / No-Code Agents (M365 Copilot & Agent Builder)

Configuration-driven assistants scoped to SharePoint, OneDrive, and Teams data defined via `declarativeAgent.json`.

- **How It Works**: Built using natural language or form fields inside Teams, SharePoint, or M365 Copilot Chat. It wraps Microsoft 365 Copilot with custom system instructions (up to 8,000 characters) and designated corporate data sources.
- **Exact Output Metrics Available**:
- `total_active_users`: Number of users actively interacting with the agent.
- `assigned_licenses`: Total seats provisioned for the organization.
- `last_activity_date`: Timestamp of the last prompt submitted per user.

- `app_usage_breakdown`: Which app was used (Teams Copilot, Word Copilot, Excel Copilot, SharePoint).
- *(Note: Individual prompt content and raw token counts are strictly masked by Microsoft for enterprise privacy and compliance).*
- **Who Has Access to See It**:
- Global Admin, Reports Reader, or Usage Summary Reports Reader in Microsoft 365 Admin Center (`admin.microsoft.com`).
- **Exposed API / Endpoint**:
- **Microsoft Graph REST API (Reporting)**:
  `http GET https://graph.microsoft.com/v1.0/reports/getMicrosoft365CopilotUsageUserDetail(period='D30')`
  `GET https://graph.microsoft.com/v1.0/reports/getMicrosoft365CopilotUserCountTrend(period='D30')`
- **Credentials & Permissions Needed**:
- Microsoft Entra ID (Azure AD) **App Registration**.
- `Tenant ID`, `Client ID`, `Client Secret`.
- Application Permission: `Reports.Read.All` (requires Admin Consent from Company Global Admin).
- **Per-User Granularity**:
- **Yes**: Returns User Principal Name (e.g. `employee@company.com`), **UNLESS** the Microsoft 365 tenant has "*Conceal user, group, and site names in all reports*" enabled in Org Settings. If enabled, the admin must toggle it off to see real employee emails instead of anonymized GUIDs.

Pathway 2: Low-Code Workflow Agents (Microsoft Copilot Studio)

Autonomous chatbots and transactional business workflow agents built on Power Platform / Dataverse.

- **How It Works**: Built using Copilot Studio's visual canvas. Connects to enterprise systems via Power Automate connectors, triggers automated actions, and manages multi-turn decision logic.
- **Exact Output Metrics Available**:
- `total_sessions`: Total conversation sessions initiated.
- `session_outcomes`:
  - **Resolved**: Problem successfully handled by AI.
  - **Escalated**: Transferred to a human specialist.
  - **Abandoned**: User dropped off before completion.
- `topic_trigger_count`: Which specific business workflow was triggered (e.g., `Reset_Password`, `Hardware_Check`, `Order_Lookup`).
- `csat_score`: Customer/Employee Satisfaction score surveys (1 to 5 stars).
- `conversation_transcript`: Complete audit trail of user messages and tool actions in structured JSON format.
- **Who Has Access to See It**:

- Power Platform System Administrator, Environment Admin, or Bot Author in Copilot Studio Portal (`copilotstudio.microsoft.com`).
- **Exposed API / Endpoint**:
- **Microsoft Dataverse OData Web API**: `http GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/conversationtranscripts GET https://<org-env>.api.crm.dynamics.com/api/data/v9.2/bots`
- **Credentials & Permissions Needed**:
- Entra App Registration added as an **Application User** in Power Platform Admin Center.
- Role: `System Customizer` or Read permissions on `conversationtranscript`.
- `Tenant ID`, `Client ID`, `Client Secret`, and `Dataverse Environment URL`.
- **Per-User Granularity**:
- **Conditional**: For internal corporate bots authenticated via Teams/SSO, user identity (`From.Name`, `From.Id`) is logged in the transcript. For anonymous public portal bots, it logs session GUIDs.

Pathway 3: Code-First Custom Agents (Azure AI Foundry / Agent Service / Semantic Kernel)

Full-stack software agents developed by software engineers in Python or C# deployed to Azure cloud infrastructure.

- **How It Works**: Built using Semantic Kernel, AutoGen, or Azure AI Foundry Agent Service. The custom code handles prompt orchestration, memory state, and calls to external databases/APIs.
- **Exact Output Metrics Available**:
- `tokens_in` / `tokens_out`: Exact prompt and completion token counts per execution.
- `cost_usd`: Direct financial cost calculated from Azure OpenAI deployment rates.
- `latency_ms`: Execution duration across LLM inference, vector search, and tool execution.
- `tool_call_metrics`: Success/failure status of external APIs called.
- `evaluation_scores`: Automated Groundedness, Relevance, and Safety evaluations.
- **Who Has Access to See It**:
- Azure Subscription Owner, Contributor, or Monitoring Reader on the Resource Group.
- **Exposed API / Endpoint**:
- **Azure Application Insights REST API (Kusto / KQL)**: `http POST https://api.applicationinsights.io/v1/apps/{app-id}/query`
- **Azure Cost Management API**: `http POST https://management.azure.com/subscriptions/{subId}/providers/Microsoft.CostManagement/query?api-version=2023-11-01`
- **Credentials & Permissions Needed**:
- Azure Service Principal (Entra App Registration) with `Monitoring Reader` + `Cost Management Reader`.

- `Subscription ID`, `Resource Group`, `Tenant ID`, `Client ID`, `Client Secret`.
- **Per-User Granularity**:
- **100% Controllable via Code**: Azure OpenAI does not automatically know end-user emails unless our agent code explicitly attaches `user_email: "employee@company.com"` in the run metadata or OpenTelemetry baggage attributes.

Pathway 4: Governed Data Agents (Microsoft Fabric Data Agents)

Specialized analytics agents built over Fabric OneLake, Lakehouses, and Power BI semantic models.

- **How It Works**: Interprets natural language questions into DAX/SQL queries directly over company data estates.
- **Exact Output Metrics Available**:
- `capacity_units_consumed (CU Seconds)`: Processing compute billed against Fabric capacity.
- `query_throughput`: Count of business questions processed.
- `audit_events`: Which user queried which Lakehouse/Warehouse table.
- **Who Has Access to See It**:
- Fabric Capacity Administrator, Workspace Admin.
- **Exposed API / Endpoint**:
- **Fabric REST API & Power BI Activity Events API**: `http GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId} GET https://api.powerbi.com/v1.0/myorg/admin/activityevents`
- **Credentials & Permissions Needed**:
- Entra App Registration with Fabric Admin permission `Tenant.Read.All`.
- **Per-User Granularity**:
- **Yes**: Full user principal names (`UserId: clerk@company.com`) are logged in the activity audit stream.

---

2. Enterprise AI Subscriptions (Fixed Seats & Developer Tools)

Enterprises license developer and knowledge-worker AI primarily through **fixed seat subscriptions**:

Subscription Tier	Enterprise Price	Telemetry Source	Key Output Metrics
GitHub Copilot Business	**$19 / user / mo**	GitHub REST API (`/orgs/{org}/copilot/...`)	`total_suggestions`, `total_acceptances`, `acceptance_rate_%`, `lines_suggested`, `lines_accepted`, `last_activity_at`, IDE type (`vscode`, `jetbrains`).
GitHub Copilot Enterprise	**$39 / user / mo**	GitHub REST API (`/enterprises/{ent}/copilot/...`)	Everything in Business + `pr_summaries_created`, custom organization knowledge base queries.
Microsoft 365 Copilot	**$30 / user / mo**	Microsoft Graph Reports API	Active vs. Inactive users, app breakdown (Teams, Word, Excel, Outlook), last active date per employee.
Microsoft Copilot Studio	**$200 / tenant / mo** *(25k msgs)*	Dataverse OData API	Total sessions, resolved vs escalated sessions, CSAT ratings.

Key Metrics from the Official GitHub Copilot Metrics API:

- `copilot_ide_code_completions.total_suggestions_count`: Total code suggestions shown to developers.
- `copilot_ide_code_completions.total_acceptances_count`: Total code suggestions accepted by developers.
- `acceptance_rate`: Percentage of code recommendations accepted and kept by developers:
- **Formula**: `(Total Suggestions Accepted / Total Suggestions Shown) * 100`
- *Example*: If Copilot suggests 1,000 code completions and developers accept 280 of them, the Acceptance Rate is **28.0%** (industry benchmark is 25% to 35%).
- `total_lines_suggested` **vs** `total_lines_accepted`: Volume of code authored by AI and kept by the developer.
- `last_activity_at`: Exact timestamp of last active trigger per employee (used to detect inactive seats and shelfware waste).

---

3. Practical Implementation Strategy With Our Current Resources

> **Architecture & Ingestion Note**: Based on the raw output metrics received from each telemetry source, we will design the data ingestion pipelines and write the backend parsing logic to extract, transform, and aggregate the required data points into our dashboard views (Scorecard, Cost Governance, and Seats & Adoption).

3.1 Fixed Seats Telemetry: What We Need From Balaji (Admin)

To show real employee adoption and inactive license savings in the dashboard, we only need from Balaji:
* **Microsoft 365 Copilot Usage CSV Export** from the M365 Admin Center (`admin.microsoft.com → Reports → Usage → Microsoft 365 Copilot → Export`), **OR**
* **Azure Entra App Registration credentials** (`Tenant ID` , `Client ID` , `Client Secret`) with `Reports.Read.All` read permission.

This gives us the real list of licensed company employees, active usage dates, and inactive seats to show immediate cost-optimization savings.

3.2 Custom Autonomous Agents: Live Demonstration via Azure AI Foundry

We can build a simple, realistic enterprise agent that connects directly to an Azure AI Foundry / Azure OpenAI LLM resource to demonstrate live employee interactions and real usage metrics in real time:

1. **Live Employee Query Execution**:
2. The employee interacts with the agent to solve real workplace requests (e.g. IT device troubleshooting, access requests).
3. The query is processed live by the Azure AI Foundry LLM deployment.
4. **Real-Time Telemetry Extracted**:
5. **Employee Attribution**: Captured directly from the authenticated session (`user_email`).
6. **Exact Token Consumption**: Direct prompt tokens and completion tokens returned by Azure OpenAI.
7. **Actual Run Cost**: Real financial spend per execution based on Azure model deployment pricing.
8. **Handling Time Saved**: Standard manual task duration saved by auto-resolving the request (e.g. 25 minutes saved).
9. **Why This Works Best for Demos**:
10. It provides 100% verified, live results showing real employee usage without synthetic estimates.

---

4. How to Identify What Type of Agents Are in Your Stakeholder's GitHub Repo

When your stakeholder shares a GitHub repository containing AI agents, inspect its file tree to immediately determine its architecture:

```
Stakeholder GitHub Repository
│
├── CASE A: Declarative Agent (M365 Copilot Extension)
│   ├── appPackage/
│   │   ├── manifest.json
│   │   └── declarativeAgent.json   <-- Look for this manifest file!
│   └── (No executable backend code; configuration package only)
│
├── CASE B: Copilot Studio Solution (Power Platform Unpack)
│   ├── Solution.xml
│   ├── bots/                       <-- Look for bots & botcomponents folders!
│   └── botcomponents/
│   └── (Exported low-code solution; runs in Dataverse)
│
└── CASE C: Code-First Custom Agent (Foundry / Semantic Kernel)
    ├── requirements.txt / pyproject.toml
    ├── semantic_kernel/ or azure.ai
    ├── plugins/
    └── main.py / agent.py          <-- Full Python/C# execution logic!
```

Action Steps Based on Repo Structure:

1. **If Case A (Declarative Agent)**:
2. Do not run the repo code. It is an M365 manifest.
3. Ask the admin for an Entra App Registration to pull usage data from the **Microsoft Graph Reports API**.
4. **If Case B (Copilot Studio Export)**:
5. It is a Power Platform solution.
6. Connect to their **Dataverse Web API** (`conversationtranscripts`) to ingest session outcomes.
7. **If Case C (Code-First Custom Agent)**:
8. It is custom software running on Azure.
9. Connect to their **Azure Application Insights** or configure a lightweight webhook in their agent loop.

Summary Checklist to Ask Your Stakeholder:

1. "*Which platform runs these agents in production: **M365 Copilot**, **Copilot Studio**, or **Azure AI Foundry**?*"
2. "*Can IT provide an **Azure Entra App Registration** with `Reports.Read.All` (for M365 Copilot) or read access to the **Dataverse Web API / Application Insights**?*"
